

Checklist para la entrega de la práctica de JDBC

Esta checklist no es una rúbrica ni un contrato de evaluación con la que podrás determinar de forma precisa la nota de tu entrega. Es simplemente una lista de los errores más frecuentes en estas entregas. La intención es que la uses antes de subir tu proyecto al campus.

Los chequeos de errores que se comentan a continuación son algunos de los más frecuentemente detectados a lo largo de varios cursos, pero no son todos. No tener ninguno de estos no te garantiza que tu entrega sea perfecta, pero sí será bastante mejor que si no la revisas.

Precondiciones
Las interfaces de la capa de servicio no han sido modificadas Los tests de aceptación originales se compilan y ejecutan correctamente Estas precondiciones deben cumplirse para que la entrega no sea descartada. Las interfaces deben ser respetadas tal cual se han entregado. Constituyen un contrato entre el cliente y el proveedor del servicio. Ten en cuenta que un script copiará las interfaces y los tests originales en tu entrega y después ejecutará los tests. Por lo que las modificaciones que hayas hecho a tus tests e interfaces se perderán y se generarán errores de compilación y/o ejecución, lo que invalidará la entrega. Antes de hacer tu entrega verifica esto, no vaya a ser que, inadvertidamente, hayas introducido algún cambio en el código de los tests o de las interfaces. La versión de Java a utilizar es 24 Modifica el fichero pom.xml para que la versión de java sea la 24.
Ejecución
La aplicación no se inicia El proyecto no compila, no arranca o no permite acceder a la interfaz principal. Algunos casos de uso de obligada implementación no se realizaron o no hacen nada Alguno de los casos de uso marcados como obligatorios no está desarrollado o no realiza ninguna acción observable La ejecución se interrumpe con excepciones La aplicación lanza errores o excepciones no controladas que impiden su funcionamiento normal en escenarios que no debería. Funciona con errores El programa se ejecuta, pero presenta fallos evidentes en la lógica o en los resultados que impiden el uso correcto de las funcionalidades principales.
Legibilidad, mantenimiento y claridad del código
No se siguen las recomendaciones de estilo y legibilidad del código El formato, la indentación o la organización del código dificultan su comprensión o mantenimiento. No se siguen las normas indicadas en los enunciados (queries.properties, nombres gateways, nombres transaction scripts, etc) Nombres incorrectos en archivos como queries.properties, gateways o transaction scripts, o estructura del proyecto distinta a la especificada. Métodos demasiado complejos o ininteligibles Se penalizarán las implementaciones con una lógica enrevesada, falta de comentarios o ausencia de modularidad que dificulte su lectura o revisión. Proyecto entregado contiene warnings No se aceptarán proyectos que, al compilar, generen advertencias sin justificar ni corregir.
Implementación del patrón capas: acoplamiento indebido entre capas
Existen dependencias de capas superiores Una capa inferior (por ejemplo, de persistencia o servicios) depende de clases o componentes definidos en capas superiores (como la interfaz de usuario o la lógica de aplicación), no respetando así la arquitectura vista en clase. Existen dependencias de otros subsistemas de la misma capa

Un componente accede directamente a otro módulo de igual nivel, rompiendo la independencia interna de la capa.
Implementación del patrón capas: violación de responsabilidades por capa
Código de presentación fuera de la capa UI Elementos relacionados con la interfaz o la interacción con el usuario aparecen en otras capas del sistema. Código de negocio fuera de la capa de negocio La lógica de aplicación o las reglas de dominio están implementadas en la capa de presentación o en la de persistencia. Código de persistencia fuera de la capa de persistencia El acceso a bases de datos o consultas fuera de los componentes designados para ello.
Implementación del patrón DTO
Código de negocio o persistencia en un assembler Los assembler deben limitarse a la conversión entre objetos y no contener lógica de negocio ni operaciones de acceso a datos. Transformaciones innecesarias entre objetos dto (S11) Se penalizará la creación o conversión redundante de DTOs que no aporte valor funcional o estructural. Los dto de la capa de persistencia no son una vista de la base de datos (P16) Los objetos de transferencia Record deben reflejar fielmente la información almacenada, evitando divergencias entre el modelo persistente y los datos reales.
Implementación del patrón Fachada
Fachada de servicios con lógica, fuera de lugar (S1) La fachada debe actuar únicamente como punto de entrada a la capa de negocio, sin contener lógica adicional.
Implementación del patrón Factoría
Los objetos command no utilizan la factoría de persistencia para obtener los gateways (S12) Cada comando debe acceder a los recursos de persistencia exclusivamente a través de la factoría correspondiente. Los objetos de la capa presentación no utilizan la factoría de service para obtener las fachadas (A2) Las clases cliente deben obtener las fachadas mediante la factoría definida, no instanciarlas directamente.
Implementación de los patrones TS y Command
Transaction scripts no implementan la interfaz Command Comandos no ejecutados utilizando CommandExecutor Todas las operaciones deben canalizarse por el CommandExecutor para asegurar consistencia y trazabilidad. Los objetos Gateway crean y liberan conexiones y/o transacciones Algún comando no se ajusta al Javadoc de la interfaz (S9)
Implementación del patrón TDG
Clases TDG con lógica de negocio Los TDG deben limitarse a realizar accesos a la tabla correspondiente Interfaces TDG no extienden interfaz Gateway Interfaces TDG añaden métodos similares a los que ya existen en Gateway, o métodos que no implementan ni se usan Cada TDG debe contener un único método para añadir, borrar o modificar. Cualquiera otro que intente hacer operaciones parciales violará el patrón Interfaces TDG añaden métodos de recuperación de información demasiado simples.

Elección del tipo de método en el TDG Opción A – Método específico (p. ej., <code>checkWorkOrderIsFinished(id)</code>): Adecuado cuando solo se necesita comprobar una condición simple. Evita tener que realizar la transformación entre <code>ResultSet</code> y objeto <code>Record</code> y mejora la eficiencia de la consulta. Opción B – Método genérico (p. ej., <code>findById(workOrderId)</code>): Recomendable cuando se requiere más información del objeto o cuando la lógica de negocio asociada es compleja o puede cambiar. Mantiene la responsabilidad de negocio en la capa correspondiente y favorece la reutilización del objeto. En resumen: usa un método específico si solo necesitas un dato puntual, y recupera la entidad completa si la condición forma parte de la lógica de negocio. Interfaces TDG añaden métodos de recuperación de información demasiado generales. Por el contrario, también está mal no implementar solo un método genérico <code>findAll</code> en el gateway para luego filtrar toda la información en la capa de servicio; esto provoca ineficiencia y sobrecarga innecesaria en la aplicación. Gateway devuelve <code>Record</code> con datos ajenos a su tabla Gateways manipula varias tablas o manipula una tabla ajena de forma inconveniente Un Gateway de una tabla A pueden consultar otras tablas para decidir si una fila de la tabla A es candidata a ser retornada. Pero no para otras cosas.
Sentencias SQL
Lógica de negocio en las consultas o consultas demasiado complejas Hay consultas o parámetros de conexión no externalizados a los ficheros de propiedades
JDBC
No se liberan los recursos o se hace de forma incorrecta
Excepciones
Métodos de las clases de service lanzan excepciones SQL, Persistence o Runtime Métodos de clases de persistencia lanzan <code>BusinessException</code> Métodos de clases de persistencia lanzan <code>SQLException</code> o <code>RuntimeException</code> en lugar de <code>PersistenceException</code> catch <code>BusinessException</code>, <code>SQLException</code> o <code>RuntimeException</code> (excepto <code>CommandExecutor</code>) en clases de service catch <code>BusinessException</code>, <code>PersistenceException</code>, <code>RuntimeException</code> o <code>Exception</code> en alguna clase en un paquete action (excepto código proporcionado) catch <code>RuntimeException</code>, <code>NullPointerException</code>, <code>Exception</code> en cualquier parte del código, excepto el proporcionado. Clases de UI no pueden throw <code>new BusinessException()</code>, throw <code>new SQL</code>, throw <code>new Persistence</code>.
Excepciones
Errores básicos de java (campos no privados excepto en <code>Dto</code> , comparaciones de objetos con <code>==</code> en lugar de <code>equals</code> , etc)